

Python Essentials

NAME – Aryan Raj

REG.NO - 26BAI10559

BRANCH – CSE AI&ML

YEAR- 1ST Year

DATE- 30th September 2026

University: VIT Bhopal University

CONTENT

S. No.	CONTENT	Page no.
1	Introduction	3
2	Problem Statement	5
3	Functional Requirements	7
4	Non-functional Requirements	16
5	System Architecture	20
6	Design Diagrams	26
7	Design Decisions & Rationale	32
8	Implementation Details	37
9	Screenshots and Results	44
10	Testing Approach	57
11	Challenges Faced	62
12	Learnings & Key Takeaways	65
13	Future Enhancements	68
14	References	71

INTRODUCTION

EduTrack is a Python-based command-line application designed to analyze and organize student academic information. The system takes basic student details, subject marks, attendance information, previous academic performance, and backlog details as input.

The application processes this information using programming and problem-solving techniques to calculate subject grades, grade points, SGPA, CGPA, attendance percentage, eligibility status, academic risk, and other performance indicators.

EduTrack also provides a CGPA goal and performance simulator, academic explanations, recommendations, an academic dashboard, and semester-wise academic history.

The project is developed for the CSE1021 – Introduction to Problem Solving and Programming

course and applies concepts such as algorithms, functions, conditional statements, loops, lists, dictionaries, sorting, numerical calculations, input validation, and file handling.

The application uses JSON for local storage and is designed to run completely through the command line without requiring external Python packages.

PROBLEM STATEMENT

Students receive academic information in the form of subject marks, attendance, previous CGPA, and backlog records. However, manually combining these values and understanding the overall academic position can be time-consuming and may lead to calculation errors.

A student may know individual marks or attendance percentages but may not have a simple way to determine how these values affect SGPA, CGPA, attendance eligibility, academic risk, and future academic targets.

EduTrack addresses this problem by providing a single command-line application that accepts the required academic information

and processes it using structured algorithms and Python programming techniques.

The system calculates academic performance, analyzes attendance, identifies academic risk factors, explains the generated results, provides recommendations, simulates future CGPA scenarios, and maintains semester-wise academic records.

The project focuses on providing a simple, structured, and understandable approach to academic analysis while applying the problem-solving and programming concepts covered in CSE1021 – Introduction to Problem Solving and Programming.

FUNCTIONAL REQUIREMENTS

The functional requirements define the operations and services that EduTrack provides to the user.

1. Student Profile Management

The system shall allow the user to enter:

- Student Name
- Registration Number
- Semester

The entered information shall be used to associate the academic record with the student.

2. Academic Information Input

The system shall allow the user to enter:

- Marks for Physics
- Marks for Chemistry
- Marks for Mathematics
- Marks for English
- Marks for UHV
- Classes attended
- Total classes
- Previous CGPA for semesters greater than 1
- Completed credits before the current semester
- Backlog information

The subject names and credits shall be predefined within the system.

3. Academic Performance Analysis

The system shall process the entered marks and calculate:

- Subject-wise grades
- Subject-wise grade points
- SGPA
- Current CGPA
- Subject ranking based on marks

The system shall also identify strong-performing subjects and subjects requiring attention.

4. Attendance Compliance and Safe-Zone Analysis

The system shall calculate the student's attendance percentage using the number of classes attended and total classes.

The system shall compare the calculated attendance with the configured attendance

threshold and display the corresponding eligibility status.

The system shall also calculate:

- . Classes required to reach the attendance threshold
- . Maximum additional absences within the configured safe zone

5. CGPA Goal and Performance Simulation

The system shall allow the user to enter a target CGPA.

The system shall:

- . Calculate the SGPA required to reach the target when possible.
- . Identify when the target cannot be reached within one semester under the configured credit structure.

- Display projected CGPA values for different future SGPA scenarios.

6. Academic Risk Analysis

The system shall analyze academic factors including:

- Attendance status
- Backlogs
- Current SGPA
- Subjects requiring attention

Based on the identified factors, the system shall classify the academic risk level as:

- Low
- Moderate
- High

7. Academic Performance Explanation

The system shall explain the factors contributing to the calculated academic results.

The explanation shall consider:

- Attendance percentage
- Attendance status
- SGPA
- Strong subjects
- Subjects requiring attention
- Backlog information
- Overall academic risk
-

8. Academic Recommendations

The system shall generate recommendations based on the student's academic information and analysis.

Recommendations may address:

- Attendance
- Subject performance
- Backlogs
- Overall academic performance
- Identified academic risk factors

9. Academic Dashboard

The system shall provide a consolidated dashboard containing:

- Student information
- SGPA
- Current CGPA

- Attendance percentage
- Attendance status
- Academic risk level
- Subject-wise performance
- Strong subjects
- Subjects requiring attention
- Backlogs
- Attendance safe-zone information

10. Academic History

The system shall store academic records in JSON format.

The system shall allow previously stored records to be retrieved using the student's registration number and display the available semester-wise academic information.

11. Input Validation and Error Handling

The system shall validate user inputs including:

- Semester values
- Subject marks
- Classes attended
- Total classes
- Previous CGPA
- Backlog information
- Menu selections

The system shall reject invalid inputs and request valid values from the user.

The system shall also prevent inconsistent attendance input, such as the number of attended classes being greater than the total number of classes.

NON-Functional Requirements

The non-functional requirements define how EduTrack should operate in terms of usability, reliability, performance, maintainability, and resource usage.

1. Usability

The system shall provide a simple command-line interface with clear menus, prompts, and output messages so that users can understand and operate the application without requiring advanced technical knowledge.

2. Performance

The system shall process academic information and generate results within a short amount of time. Calculations such as SGPA, CGPA, attendance percentage, risk analysis, and

recommendations shall be performed without unnecessary processing.

3. Reliability

The system shall produce consistent results when valid academic information is provided. Stored academic records shall remain available for later retrieval unless the underlying JSON data is removed or changed.

4. Error Handling

The system shall handle invalid user input without terminating unexpectedly. Appropriate messages shall be displayed when the user enters invalid marks, CGPA values, semester values, attendance information, or menu selections.

5. Maintainability

The application shall use separate Python modules for different responsibilities such as academic analysis, attendance analysis, CGPA

simulation, risk analysis, recommendations, dashboard generation, and data storage.

This modular structure shall make the project easier to understand, test, modify, and extend.

6. Resource Efficiency

The system shall use minimal system resources because it operates as a command-line application and uses local JSON storage instead of requiring a database server or external services.

7. Portability

The application shall be executable on systems that support Python 3 and a command-line terminal, including Windows, Linux, and macOS.

8. Data Integrity

The system shall validate important input values before processing them. Attendance values shall be checked for consistency, marks shall remain within the valid range, and CGPA values shall remain within the configured range.

9. Scalability

The modular design shall allow additional subjects, academic metrics, simulation scenarios, and analysis features to be added without requiring a complete redesign of the application.

SYSTEM ARCHITECTURE

EduTrack follows a modular architecture in which different Python modules are responsible for specific tasks. The system accepts academic information through a command-line interface, processes the information using separate analysis modules, stores academic records in a local JSON file, and displays the generated results to the user.

The architecture consists of the following major components:

1. User Interface

The user interacts with EduTrack through the command-line interface. The interface provides the main menu and collects student and academic information.

2. Input Modules

The input modules collect and validate:

- Student details
- Registration number
- Semester
- Subject marks
- Attendance information
- Previous CGPA
- Completed credits
- Backlog information
-

3. Academic Analysis Modules

The academic analysis modules process the collected information and perform:

- Grade calculation
- Grade point calculation
- SGPA calculation

- CGPA calculation
- Subject ranking
- Strong and weak subject identification

4. Attendance Analysis Module

The attendance module calculates attendance percentage and determines the student's attendance status. It also calculates the classes required to reach the configured attendance threshold and the maximum additional absences within the safe zone.

5. CGPA Simulation Module

The CGPA simulation module calculates the SGPA required to reach a target CGPA and generates projected CGPA values for different future SGPA scenarios.

6. Academic Risk Analysis Module

The risk analysis module examines attendance, backlogs, SGPA, and subjects requiring attention to identify academic risk factors.

7. Explanation and Recommendation Modules

The explanation module provides reasons for the calculated academic results. The recommendation module generates suggestions based on the student's academic conditions.

8. Dashboard Module

The dashboard module combines the processed information and presents a consolidated view of the student's academic performance, attendance, CGPA, risk level, and academic highlights.

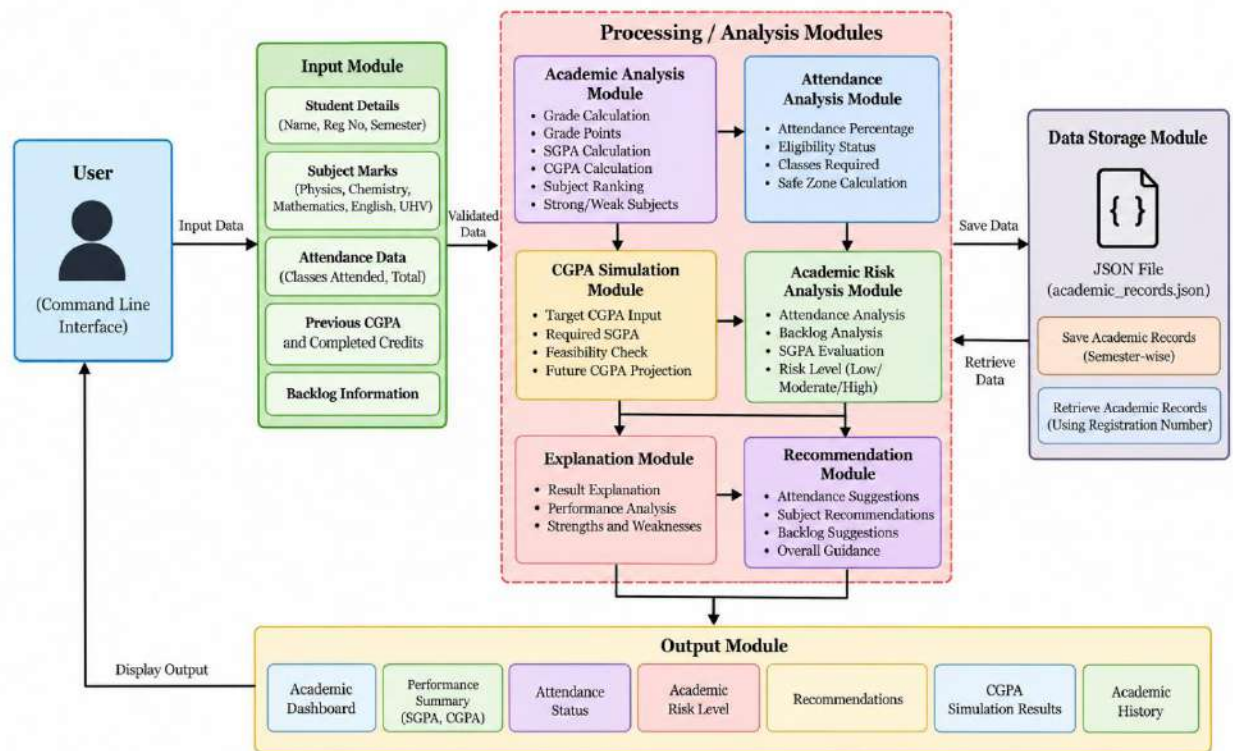
9. Data Storage Module

The storage module uses a local JSON file to save and retrieve academic records. Student records can be retrieved using the registration number, allowing semester-wise academic history to be maintained.

10. Output

The processed results are displayed through the command-line interface. The output includes academic performance, attendance status, risk analysis, recommendations, CGPA simulation results, dashboard information, and academic history.

11. Overall Architecture Flow



DESIGN DIAGRAMS

1. Use Case Diagram

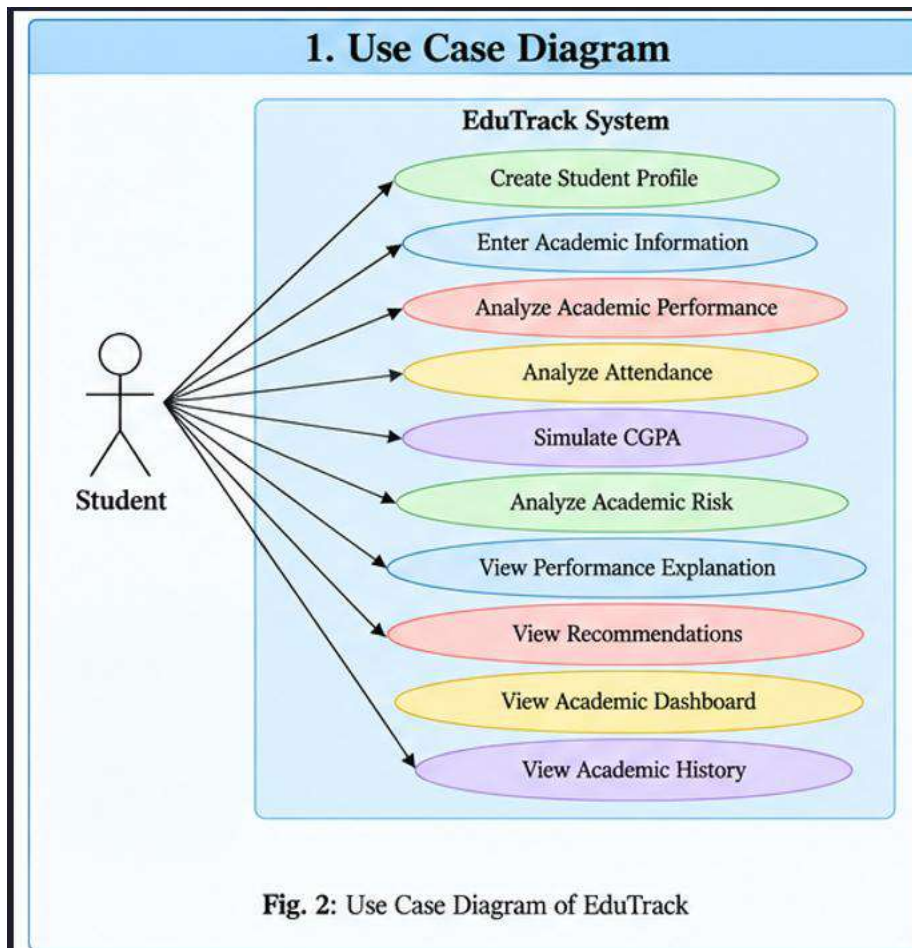
The Use Case Diagram represents the main interactions between the student and the EduTrack system.

Main Actor: Student

Use Cases:

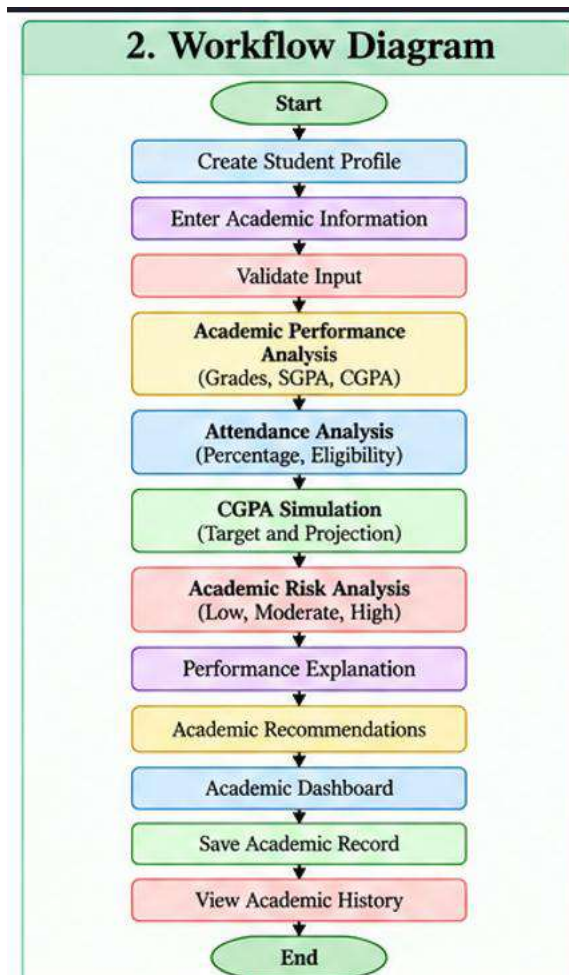
- Create Student Profile
- Enter Academic Information
- Analyze Academic Performance
- Analyze Attendance
- Simulate CGPA
- Analyze Academic Risk
- View Performance Explanation

- View Recommendations
- View Academic Dashboard
- View Academic History



2. Workflow Diagram

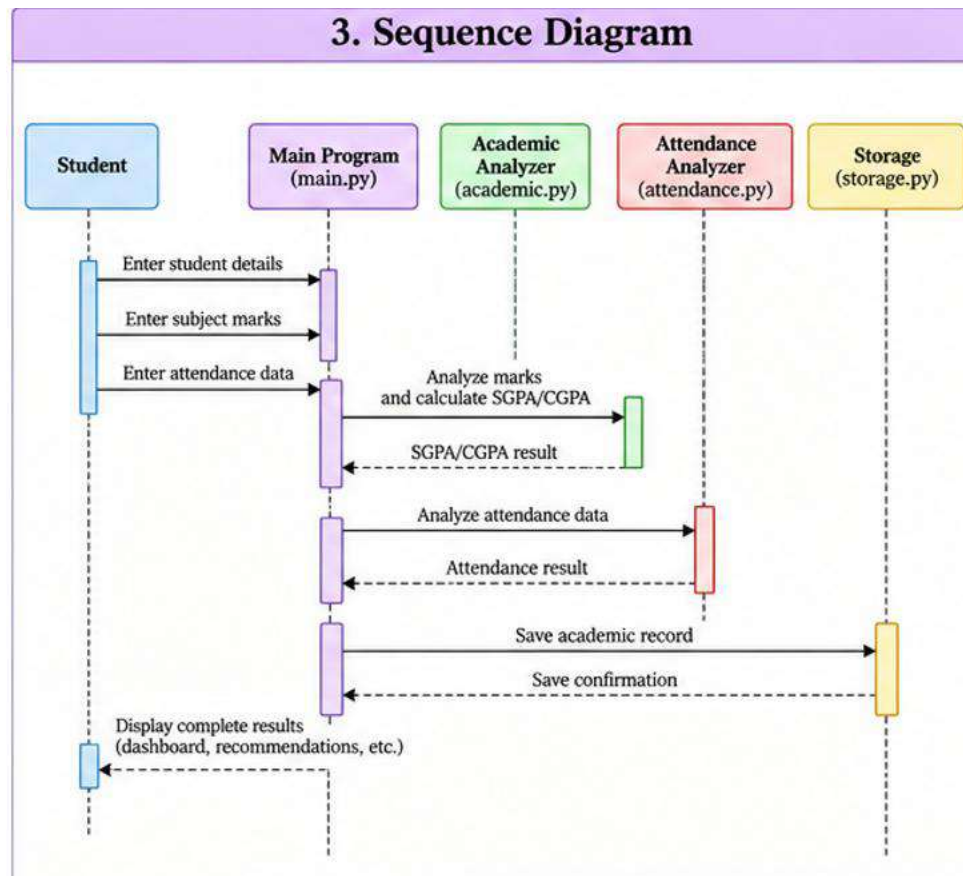
The Workflow Diagram represents the sequence of operations followed by the user while using EduTrack.



3. Sequence Diagram

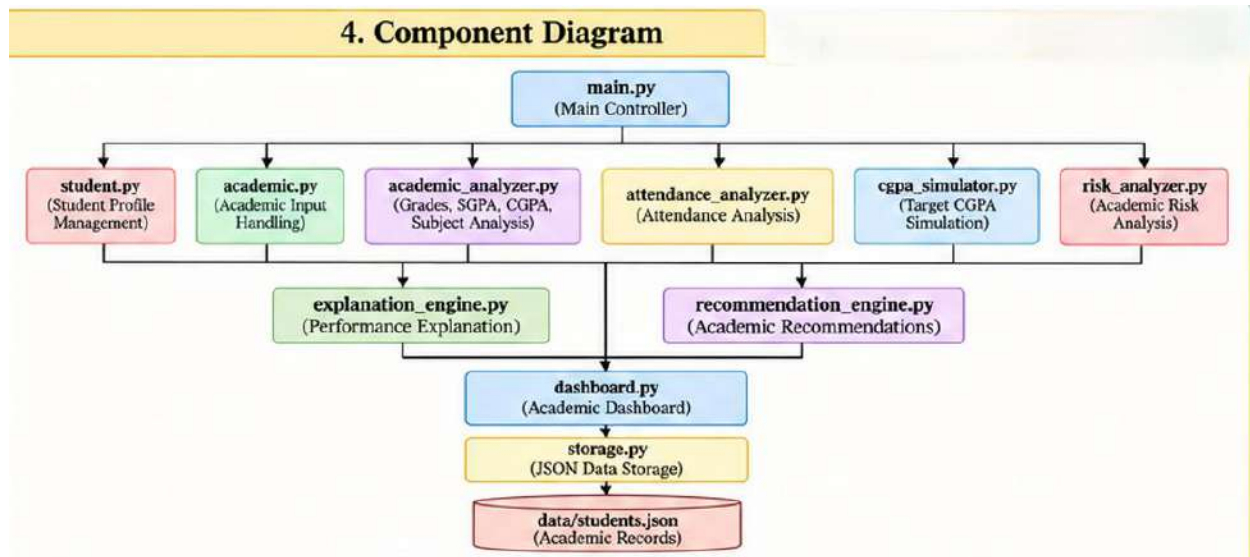
The Sequence Diagram represents the interaction between the student, EduTrack modules, and JSON storage during a typical academic analysis

process.



4. Class / Component Diagram

The project is implemented using multiple Python modules, with each module handling a specific responsibility.

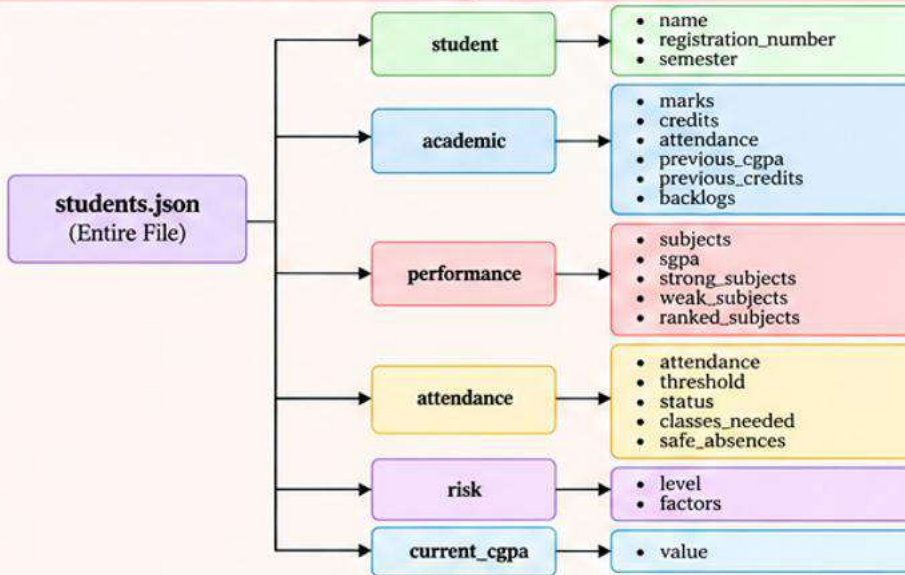


5. Storage Design

EduTrack uses a JSON file for local academic record storage instead of a relational database.

The stored record contains the following major sections:

5. JSON Storage Structure



DESIGN DECISIONS AND RATIONALE

The design of EduTrack was based on simplicity, modularity, maintainability, and alignment with the concepts covered in CSE1021 – Introduction to Problem Solving and Programming.

1. Python as the Programming Language

Python was selected because the course focuses on Python programming and its fundamental programming concepts. It also provides simple syntax for implementing calculations, conditional logic, loops, functions, lists, dictionaries, and file handling.

2. Command-Line Interface

A command-line interface was selected to keep the application simple and fully executable

through a terminal. It allows the complete project workflow to be demonstrated without requiring additional graphical frameworks.

3. Modular Program Structure

The application is divided into multiple Python files, with each module responsible for a specific function. This separates student management, academic analysis, attendance analysis, CGPA simulation, risk analysis, recommendations, dashboard generation, and storage

.

4. Predefined Subject Credits

Subject names and credits are predefined in the application so that the user only needs to enter marks. This reduces unnecessary input and ensures that the configured credit values are used consistently during SGPA and CGPA calculations.

5. JSON-Based Storage

JSON was selected for storing academic records because it provides a simple structured format that can be handled directly using Python's built-in file and JSON functionality. It also avoids the need for an external database server.

6. Input Validation

Input validation was included to prevent invalid values from being processed. Marks, CGPA, semester values, attendance information, and menu selections are checked before further processing.

7. Separate Analysis Modules

Different analysis tasks are implemented in separate modules. This allows each part of the application to be tested and modified

independently and makes the overall program easier to maintain.

8. Academic Simulation

The CGPA simulator was included to apply numerical calculations to a practical academic problem. It allows users to examine the relationship between current academic performance, future SGPA, and target CGPA.

9. Academic History

Academic history was included so that the application is not limited to a single calculation session. Saved semester records can be retrieved using the registration number and reviewed later.

10. Course Alignment

The project design directly applies concepts from CSE1021, including problem decomposition, algorithms, functions, conditional statements, loops, lists, dictionaries, sorting, numerical calculations, input validation, and file handling.

IMPLEMENTATION DETAILS

EduTrack is implemented as a modular Python command-line application. Each major function of the system is separated into an individual Python module.

1. Student Profile Module

The `student.py` module handles student profile creation. It collects the student's name, registration number, and semester and validates the semester value.

2. Academic Input Module

The `academic.py` module manages academic information input. It stores the predefined subjects and credits and collects subject marks,

attendance information, previous CGPA, completed credits, and backlog details.

3. Academic Analysis Module

The `academic_analyzer.py` module calculates grades and grade points from the entered marks. It uses the configured subject credits to calculate SGPA and current CGPA. It also identifies strong and weak subjects and ranks subjects according to marks.

4. Attendance Analysis Module

The `attendance_analyzer.py` module calculates attendance percentage using the number of classes attended and total classes. It determines the attendance status and calculates the classes required to reach the configured attendance threshold and the maximum additional absences within the safe zone.

5. CGPA Simulation Module

The `cgpa_simulator.py` module performs CGPA-related numerical calculations. It calculates the SGPA required for a target CGPA and generates projected CGPA values for different future SGPA scenarios.

6. Academic Risk Analysis Module

The `risk_analyzer.py` module evaluates academic conditions using attendance, backlogs, SGPA, and subjects requiring attention. It identifies risk factors and classifies the overall risk level as Low, Moderate, or High.

7. Explanation Module

The `explanation_engine.py` module generates explanations for the academic results. It considers attendance, SGPA, subject

performance, and backlog information to explain the result shown to the user.

8. Recommendation Module

The `recommendation_engine.py` module generates recommendations based on the student's academic conditions. The recommendations focus on areas such as attendance, subject performance, backlogs, and overall academic improvement.

9. Dashboard Module

The `dashboard.py` module presents the major academic results in a consolidated format. It displays student information, SGPA, CGPA, attendance, attendance status, academic risk, subject performance, backlogs, and safe-zone information.

10. Storage Module

The `storage.py` module manages local JSON storage. It creates the required data file when necessary, saves academic records, updates existing semester records, and retrieves academic history using the registration number.

11. Main Controller

The `main.py` module controls the complete application workflow. It displays the main menu, manages user choices, calls the required modules, prepares academic analysis, displays results, and coordinates the saving and retrieval of academic records.

12. Data Storage

Academic records are stored in:

`data/students.json`

The JSON structure contains student information, academic information, performance results, attendance analysis, risk analysis, and current CGPA.

13. Programming Concepts Applied

The implementation uses the following CSE1021 concepts:

- Variables and data types
- Expressions and statements
- Functions
- Parameters and arguments
- Conditional statements
- Loops
- Lists
- Dictionaries
- Sorting

- Numerical calculations
- Input validation
- File handling
- JSON data representation
- Modular programming
-

14. Error Handling

The application uses validation loops and exception handling to manage invalid user input. The program prevents invalid marks, CGPA values, semester values, and inconsistent attendance values from being processed.

15. Execution

The complete application can be executed from the command line using: `python main.py`

SCREENSHOTS AND RESULTS

The EduTrack application was executed through the command-line interface and tested with different academic inputs. The following screenshots demonstrate the main features and outputs of the system.

1. Main Menu

The main menu provides access to all major functions of EduTrack, including student profile creation, academic analysis, attendance analysis, CGPA simulation, risk analysis, recommendations, dashboard, and academic history.

```
=====
                        EDUTRACK
        Student Academic Performance & Eligibility Analyzer
=====

MAIN MENU
-----
1. Create Student Profile
2. Enter Academic Information
3. Academic Performance Analysis
4. Attendance Compliance & Safe-Zone Analyzer
5. CGPA Goal & Performance Simulator
6. Academic Risk Analysis
7. Academic Performance Explanation
8. Academic Recommendations
9. Academic Dashboard
10. Academic History
0. Exit
-----
Enter your choice: 1

STUDENT PROFILE
-----
Student Name: ABC
Registration Number: 26BAI1234
Semester: 1

Student profile created successfully.
-----
Name: ABC
Registration Number: 26BAI1234
Semester: 1
```

2. Student Profile Creation

The student profile module accepts the student's name, registration number, and semester.

```
MAIN MENU
-----
1. Create Student Profile
2. Enter Academic Information
3. Academic Performance Analysis
4. Attendance Compliance & Safe-Zone Analyzer
5. CGPA Goal & Performance Simulator
6. Academic Risk Analysis
7. Academic Performance Explanation
8. Academic Recommendations
9. Academic Dashboard
10. Academic History
0. Exit
-----
Enter your choice: 2

ACADEMIC INFORMATION
-----

MARKS
-----
Physics: 94
Chemistry: 92
Mathematics: 91
English: 57
UHV: 85

ATTENDANCE
-----
Classes Attended: 54
Total Classes: 60
Backlogs? (Y/N): N

Academic information saved successfully.
```

3. Academic Information Input

The academic input module collects subject marks, attendance information, previous CGPA

where applicable, completed credits, and backlog information.

MAIN MENU

- 1. Create Student Profile
- 2. Enter Academic Information
- 3. Academic Performance Analysis
- 4. Attendance Compliance & Safe-Zone Analyzer
- 5. CGPA Goal & Performance Simulator
- 6. Academic Risk Analysis
- 7. Academic Performance Explanation
- 8. Academic Recommendations
- 9. Academic Dashboard
- 10. Academic History
- 0. Exit

Enter your choice: 3

ACADEMIC PERFORMANCE ANALYSIS

Physics	Marks: 94.0	Grade: S	Grade Point: 10
Chemistry	Marks: 92.0	Grade: S	Grade Point: 10
Mathematics	Marks: 91.0	Grade: S	Grade Point: 10
English	Marks: 57.0	Grade: D	Grade Point: 6
UHV	Marks: 85.0	Grade: A	Grade Point: 9

SGPA: 9.19

SUBJECT RANKING

- 1. Physics - 94.0
- 2. Chemistry - 92.0
- 3. Mathematics - 91.0
- 4. UHV - 85.0
- 5. English - 57.0

STRONG SUBJECTS

Physics, Chemistry, Mathematics, UHV

SUBJECTS REQUIRING ATTENTION

English

Current CGPA: 9.19

4. Academic Performance Analysis

The performance analysis displays subject-wise marks, grades, grade points, SGPA, subject ranking, strong subjects, and subjects requiring attention.

```
MAIN MENU
-----
1. Create Student Profile
2. Enter Academic Information
3. Academic Performance Analysis
4. Attendance Compliance & Safe-Zone Analyzer
5. CGPA Goal & Performance Simulator
6. Academic Risk Analysis
7. Academic Performance Explanation
8. Academic Recommendations
9. Academic Dashboard
10. Academic History
0. Exit
-----
Enter your choice: 4

ATTENDANCE COMPLIANCE & SAFE-ZONE ANALYZER
=====
Current Attendance: 90.00%
Required Threshold: 75%
Status: ELIGIBLE

Maximum Additional Absences Within Safe Zone: 12
```

5. Attendance Compliance and Safe-Zone Analysis

The attendance module displays the calculated attendance percentage, configured threshold, eligibility status, and safe-zone information.

```
MAIN MENU
-----
1. Create Student Profile
2. Enter Academic Information
3. Academic Performance Analysis
4. Attendance Compliance & Safe-Zone Analyzer
5. CGPA Goal & Performance Simulator
6. Academic Risk Analysis
7. Academic Performance Explanation
8. Academic Recommendations
9. Academic Dashboard
10. Academic History
0. Exit
-----
Enter your choice: 4

ATTENDANCE COMPLIANCE & SAFE-ZONE ANALYZER
=====
Current Attendance: 90.00%
Required Threshold: 75%
Status: ELIGIBLE

Maximum Additional Absences Within Safe Zone: 12
```

6. CGPA Goal and Performance Simulation

The CGPA simulator displays the current academic position, target CGPA, required future SGPA, and projected CGPA values for different future SGPA scenarios.

MAIN MENU

- 1. Create Student Profile
- 2. Enter Academic Information
- 3. Academic Performance Analysis
- 4. Attendance Compliance & Safe-Zone Analyzer
- 5. CGPA Goal & Performance Simulator
- 6. Academic Risk Analysis
- 7. Academic Performance Explanation
- 8. Academic Recommendations
- 9. Academic Dashboard
- 10. Academic History
- 0. Exit

Enter your choice: 5

CGPA GOAL & ACADEMIC PERFORMANCE SIMULATOR

Target CGPA: 9.5

CURRENT ACADEMIC POSITION

Current SGPA: 9.19

Current CGPA: 9.19

Target CGPA: 9.50

TARGET ANALYSIS

Required Next-Semester SGPA: 9.81

PERFORMANCE SCENARIOS

If Next SGPA = 7.0 -> Projected CGPA = 8.09

If Next SGPA = 8.0 -> Projected CGPA = 8.59

If Next SGPA = 9.0 -> Projected CGPA = 9.09

If Next SGPA = 10.0 -> Projected CGPA = 9.59

7. Academic Risk Analysis

The risk analysis module displays the identified academic risk level and the factors contributing to that classification.

```
MAIN MENU
-----
1. Create Student Profile
2. Enter Academic Information
3. Academic Performance Analysis
4. Attendance Compliance & Safe-Zone Analyzer
5. CGPA Goal & Performance Simulator
6. Academic Risk Analysis
7. Academic Performance Explanation
8. Academic Recommendations
9. Academic Dashboard
10. Academic History
0. Exit
-----
Enter your choice: 6

ACADEMIC RISK ANALYSIS
=====
Risk Level: MODERATE

IDENTIFIED FACTORS
-----
1. One or more subjects require additional academic attention.
```

8. Academic Performance Explanation

The explanation engine displays the reasons behind the calculated academic results, including attendance, semester performance, subject performance, and backlog information.

```

MAIN MENU
-----
1. Create Student Profile
2. Enter Academic Information
3. Academic Performance Analysis
4. Attendance Compliance & Safe-Zone Analyzer
5. CGPA Goal & Performance Simulator
6. Academic Risk Analysis
7. Academic Performance Explanation
8. Academic Recommendations
9. Academic Dashboard
10. Academic History
0. Exit
-----
Enter your choice: 7

ACADEMIC PERFORMANCE EXPLANATION
=====
WHY DID I GET THIS RESULT?
-----
1. Attendance is 90.00%, which meets the required 75% threshold.
2. The current SGPA of 9.19 indicates strong semester performance.
3. Strong performance was recorded in: Physics, Chemistry, Mathematics, UHV.
4. Additional attention is recommended for: English.
5. No academic backlogs have been reported.

OVERALL INTERPRETATION
-----
Some academic factors require attention. Improving the identified areas can strengthen the overall academic position.

```

9. Academic Recommendations

The recommendation engine generates academic priorities based on the student's current performance, attendance, backlogs, and identified risk factors.

MAIN MENU

1. Create Student Profile
2. Enter Academic Information
3. Academic Performance Analysis
4. Attendance Compliance & Safe-Zone Analyzer
5. CGPA Goal & Performance Simulator
6. Academic Risk Analysis
7. Academic Performance Explanation
8. Academic Recommendations
9. Academic Dashboard
10. Academic History
0. Exit

Enter your choice: 8

ACADEMIC RECOMMENDATIONS

1. Priority 2: Focus additional preparation on English.
2. Maintain strong performance in high-performing subjects while giving additional attention to weaker subjects.

10. Academic Dashboard

The dashboard provides a consolidated view of the student's academic information, including SGPA, CGPA, attendance, eligibility status, risk level, subject performance, and academic highlights.

Enter your choice: 9

=====

EDUTRACK
ACADEMIC DASHBOARD

=====

STUDENT INFORMATION

Name: ABC
Registration Number: 26BAI1234
Semester: 1

ACADEMIC SUMMARY

SGPA: 9.19
Current CGPA: 9.19
Attendance: 90.00%
Attendance Status: ELIGIBLE
Academic Risk: MODERATE

ATTENDANCE SAFE ZONE

Maximum additional absences within safe zone: 12

SUBJECT PERFORMANCE

Physics	Marks: 94.0	Grade: S	Credits: 4
Chemistry	Marks: 92.0	Grade: S	Credits: 4
Mathematics	Marks: 91.0	Grade: S	Credits: 4
English	Marks: 57.0	Grade: D	Credits: 3
UHV	Marks: 85.0	Grade: A	Credits: 1

ACADEMIC HIGHLIGHTS

Strong Subjects: Physics, Chemistry, Mathematics, UHV
Subjects Requiring Attention: English
Backlogs: None

=====

Academic record saved successfully.

11. Academic History

The academic history feature retrieves previously saved semester records using the student's registration number.

```
MAIN MENU
-----
1. Create Student Profile
2. Enter Academic Information
3. Academic Performance Analysis
4. Attendance Compliance & Safe-Zone Analyzer
5. CGPA Goal & Performance Simulator
6. Academic Risk Analysis
7. Academic Performance Explanation
8. Academic Recommendations
9. Academic Dashboard
10. Academic History
0. Exit
-----
Enter your choice: 10

ACADEMIC HISTORY
=====

Semester 1
-----
SGPA: 9.19
CGPA: 9.19
Attendance: 90.00%
Status: ELIGIBLE
```

Results

Testing demonstrated that EduTrack can successfully accept academic information, validate inputs, perform academic calculations, analyze attendance, identify risk factors, generate explanations and recommendations, simulate

CGPA outcomes, display a consolidated dashboard, and store academic records using JSON.

TESTING APPROACH

EduTrack was tested using different input conditions to verify that the system performs calculations correctly, handles invalid input, and produces the expected output.

1. Student Profile Testing

The student profile module was tested by entering valid student names, registration numbers, and semester values.

Invalid semester values were also tested to verify that the system requests a valid semester.

2. Marks Validation Testing

Subject marks were tested using valid values within the range of 0 to 100.

Invalid values such as negative marks and marks greater than 100 were entered to verify that the application rejects them.

3. Academic Calculation Testing

The academic analysis was tested to verify:

- Grade calculation
- Grade point calculation
- SGPA calculation
- CGPA calculation
- Subject ranking
- Strong subject identification
- Subjects requiring attention

4. Attendance Testing

Different attendance conditions were tested, including:

- Attendance above the configured threshold
- Attendance equal to the configured threshold
- Attendance below the configured threshold
- Different numbers of classes attended and total classes

The calculated attendance percentage and eligibility status were checked against the expected results.

5. CGPA Simulation Testing

Different current CGPA values and target CGPA values were tested.

The simulator was checked for:

- Required future SGPA calculation
- Target feasibility
- Projected CGPA calculation
- Different future SGPA scenarios

6. Backlog Testing

The application was tested with both no-backlog and backlog conditions.

The risk analysis, explanation, dashboard, and recommendation modules were checked to ensure that backlog information was reflected correctly.

7. Risk Analysis Testing

Different combinations of attendance, SGPA, backlogs, and weak subjects were tested to verify that the appropriate academic risk level and risk factors were generated.

8. Data Storage Testing

Academic records were saved to the JSON file and then retrieved using the student's registration number.

The academic history feature was tested to verify that previously saved semester records could be displayed correctly.

9. Invalid Input Testing

Invalid inputs were deliberately entered for:

- Menu selections
- Semester
- Marks
- CGPA

- Classes attended
- Total classes
- Backlog information

The system was checked to ensure that invalid inputs did not terminate the application unexpectedly.

10. Integration Testing

The complete application workflow was tested from student profile creation through academic information entry, analysis, dashboard generation, record storage, and academic history retrieval.

The testing confirmed that the individual modules work together through the main controller and produce the expected academic results.

CHALLENGES FACED

1. Designing a Modular Structure

One of the initial challenges was dividing the application into meaningful modules. The project was separated into different files so that each module could perform a specific responsibility without making the main program unnecessarily complex.

2. Managing Academic Calculations

Implementing SGPA and CGPA calculations required careful handling of subject credits and previous academic information. The calculations had to account for different semesters and credit values.

3. Attendance Calculation

Attendance analysis required handling different combinations of classes attended and total classes. The system also needed to calculate the

number of additional classes required to reach the configured threshold and the safe number of additional absences.

4. Input Validation

Users may enter incorrect values during program execution. Handling invalid marks, CGPA values, semester numbers, attendance values, and menu choices required validation and error-handling logic.

5. Maintaining Academic History

Another challenge was storing and retrieving semester-wise academic records. JSON storage was implemented so that records could be saved locally and retrieved using the registration number.

6. Connecting Multiple Modules

The different analysis modules needed to work together correctly. The main controller was responsible for coordinating data between

academic analysis, attendance analysis, risk analysis, recommendations, dashboard generation, and storage.

7. Handling Different Academic Conditions

The system had to produce meaningful results for different combinations of marks, attendance, backlogs, and academic performance. Testing these conditions helped identify and correct issues in the application logic.

LEARNINGS AND KEY TAKEAWAYS

1. Problem Solving

The project provided practical experience in breaking a real-world problem into smaller programming tasks and developing a logical solution for each task.

2. Algorithm Design

The project helped in applying algorithmic thinking to academic calculations, attendance analysis, subject ranking, risk identification, and CGPA simulation.

3. Python Programming

The project provided practical experience with Python variables, data types, functions, parameters, conditional statements, loops, lists, dictionaries, sorting, numerical calculations, and file handling.

4. Modular Programming

Developing the project using separate Python modules demonstrated how a larger application can be divided into smaller components with clearly defined responsibilities.

5. Input Validation

The project demonstrated the importance of validating user input before processing it and handling incorrect values without terminating the application.

6. Data Handling

Using JSON for local storage provided practical experience in representing, saving, updating, and retrieving structured data using Python.

7. Testing and Debugging

Testing different academic conditions helped identify logical issues and verify that individual modules and the complete application produced the expected results.

8. Software Documentation

The project provided experience in documenting requirements, system architecture, workflows, implementation details, testing, and project results.

9. Version Control

Using Git and GitHub provided practical experience in maintaining and organizing the project source code in a version-controlled repository.

10. Practical Application of CSE1021 Concepts

The project demonstrated how the concepts taught in CSE1021 – Introduction to Problem Solving and Programming can be applied to develop a practical command-line application.

FUTURE ENHANCEMENTS

1. Semester Performance Trends

The system can be extended to compare academic performance across multiple semesters and display changes in SGPA, CGPA, attendance, and subject performance.

2. Configurable Subject Structure

The application can be extended to allow different subject names and credit structures to be configured for different semesters or academic programs.

3. Academic Report Export

A future version can provide an option to export the student's academic analysis as a formatted report for easier sharing and record keeping.

4. Extended CGPA Simulation

The CGPA simulator can be expanded to support multiple future semesters and allow users to compare different long-term academic performance scenarios.

5. Detailed Historical Analysis

The academic history module can be extended to provide comparisons between semesters and identify long-term patterns in academic performance.

6. Additional Academic Metrics

Additional metrics such as average marks, credit completion, semester-wise subject comparisons, and performance percentages can be incorporated into future versions.

7. Improved User Interface

The command-line interface can later be enhanced with additional navigation and presentation features while retaining the existing core processing logic.

8. Expanded Data Management

Future versions can provide additional options for updating, exporting, and managing stored academic records.

REFERENCES

1. **VITyarthi – Build Your Own Project (BYOP): General Project Instructions and Submission Guidelines.**

VIT Bhopal University.

2. **CSE1021 – Introduction to Problem Solving and Programming: Course Syllabus.**

VIT Bhopal University.

3. **Python Software Foundation.**

Python 3 Documentation.

<https://docs.python.org/3/>

4. **GitHub Documentation.**

GitHub Documentation and Repository Management.

<https://docs.github.com/>